

Universidad Autónoma de Santo Domingo

PRIMADA DE AMERICA / Fundada el 28 de octubre de 1538

Nombre:

Wilmy.

Apellidos:

Medina Batista.

Matrícula:

100681393.

Asignatura:

Laboratorio De Lenguaje De Programación III.

Tema:

AJAX y Sus Funcionamientos.

Sección:

01.

Profesor:

Radhames Silverio González.

Fecha:

31/10/2026.

ÍNDICE

INTRODUCCIÓN 3
DESARROLLO: 4
1. ¿Qué es AJAX? 4
2. ¿Cuál es el propósito de AJAX? 4
Entre sus propósitos más importantes se encuentran: 4
3. ¿Cómo funciona AJAX en una página web? 5
De manera resumida, el proceso sigue estos pasos: 5
4. Relación entre AJAX, JavaScript, el navegador y el servidor. 6
5. Ventajas y desventajas del uso de AJAX. 7
Ventajas: 7
Desventajas: 7
6. Usos actuales de AJAX. 8
Formularios: 8
Búsquedas y filtros: 8
Validaciones: 8
Carga de datos: 8
Aplicaciones web dinámicas: 8
7. Diferencia entre una página web tradicional y una página que utiliza AJAX. 9
8. Ejemplo sencillo donde se explique el funcionamiento de AJAX. 9
Explicación del funcionamiento: 11
CONCLUSIÓN 12
REFERENCIAS BIBLIOGRÁFICAS 13

INTRODUCCIÓN

En el desarrollo de aplicaciones web, una de las necesidades más frecuentes es mostrar información actualizada sin obligar al usuario a cargar nuevamente toda la página. Por ejemplo, cuando una persona busca un producto, envía un formulario o consulta datos de una cuenta, espera recibir una respuesta rápida y continuar utilizando el sitio. Para atender estas situaciones se utiliza AJAX, una técnica que permite que el navegador se comunique con un servidor de manera asíncrona y actualice únicamente las partes necesarias de la interfaz.

El nombre AJAX proviene de Asynchronous JavaScript and XML, que en español significa JavaScript y XML asíncronos. Aunque originalmente se relacionaba con el intercambio de documentos XML, en la actualidad es común transportar datos en formatos como JSON, texto o fragmentos de HTML. Por eso, AJAX no debe entenderse como un lenguaje de programación independiente, sino como una forma de combinar tecnologías web para intercambiar información en segundo plano (MDN contributors, s. f.-a).

Esta técnica utiliza JavaScript para iniciar solicitudes HTTP desde el navegador, recibir respuestas del servidor y procesar los datos obtenidos. Entre las herramientas que permiten realizar estas operaciones se encuentran XMLHttpRequest y la API Fetch. Esta última ofrece una forma moderna de trabajar con solicitudes asíncronas mediante promesas y se considera una alternativa más flexible a XMLHttpRequest (MDN contributors, s. f.-b; MDN contributors, s. f.-c).

El propósito de esta investigación es explicar qué es AJAX, cuál es su utilidad, cómo se desarrolla la comunicación entre el navegador y el servidor, y cuáles son sus ventajas y limitaciones. También se presentan sus usos actuales, las diferencias entre una página tradicional y otra que utiliza esta técnica, y un ejemplo práctico con código para comprender cómo se solicita información y se muestra en una página sin recargarla completamente.

DESARROLLO:

1. ¿Qué es AJAX?

AJAX (Asynchronous JavaScript and XML) es una técnica de desarrollo web que permite que una página realice solicitudes a un servidor de forma asíncrona y utilice la respuesta para modificar una parte de su contenido sin tener que volver a cargar el documento completo. Su característica principal es que la comunicación con el servidor puede realizarse mientras el usuario continúa interactuando con los elementos de la página.

AJAX no es un lenguaje, una biblioteca específica ni una tecnología única. Es una combinación de recursos que trabajan en conjunto: HTML y CSS presentan la interfaz, JavaScript controla el comportamiento, las API del navegador permiten enviar solicitudes HTTP y el servidor procesa la petición y devuelve información. El navegador recibe la respuesta y JavaScript decide cómo incorporarla al documento.

Aunque las siglas incluyen la palabra XML, actualmente no es obligatorio utilizar ese formato. Las aplicaciones suelen intercambiar datos JSON porque permite representar objetos y listas de una manera sencilla. También pueden recibirse textos o contenido HTML, dependiendo de lo que necesite el proyecto. En consecuencia, el término AJAX se mantiene para describir el proceso de comunicación asíncrona, aunque la respuesta no sea XML (MDN contributors, s. f.-a).

2. ¿Cuál es el propósito de AJAX?

El propósito principal de AJAX es mejorar la interacción entre el usuario y una aplicación web, permitiendo solicitar o enviar información sin interrumpir la página que se está utilizando. En lugar de descargar nuevamente todos los elementos del sitio después de cada acción, el navegador puede pedir únicamente los datos que necesita y actualizar la sección correspondiente.

Entre sus propósitos más importantes se encuentran:

- **Actualizar partes de una página:** Permite cambiar una lista, un mensaje, una tabla o un panel sin reconstruir el documento completo.
- **Enviar información en segundo plano:** Hace posible procesar formularios y enviar datos al servidor sin una navegación completa.
- **Mejorar la respuesta de la interfaz:** Permite que el usuario continúe utilizando otras partes de la página mientras se espera la respuesta.
- **Reducir transferencias innecesarias:** En muchas situaciones se descargan solamente los datos requeridos para la actualización, en vez de volver a recibir toda la página.

- **Facilitar aplicaciones dinámicas:** Ayuda a construir interfaces que cambian según las acciones del usuario y la información disponible en el servidor.

Por estas razones, AJAX es útil en sitios donde el contenido cambia con frecuencia o donde se necesita una interacción continua. Sin embargo, su uso debe planificarse para controlar los errores, informar al usuario cuando una operación está en proceso y evitar solicitudes innecesarias.

3. ¿Cómo funciona AJAX en una página web?

El funcionamiento de AJAX se basa en una solicitud y una respuesta que se realizan sin necesidad de recargar toda la página. El proceso comienza cuando el usuario ejecuta una acción, como presionar un botón, escribir en un campo de búsqueda o enviar un formulario. JavaScript detecta esa acción y prepara una petición HTTP hacia una dirección del servidor.

La solicitud puede utilizar métodos como GET para consultar información o POST para enviar datos. El navegador realiza la comunicación de manera asíncrona, por lo que el resto de la interfaz puede seguir funcionando mientras llega la respuesta. Cuando el servidor recibe la petición, ejecuta la operación correspondiente y devuelve un resultado, que puede ser JSON, texto, XML o HTML.

Después de recibir la respuesta, JavaScript comprueba si la operación fue satisfactoria, interpreta los datos y actualiza el elemento apropiado del documento mediante el DOM. Si ocurre un problema de conexión o el servidor devuelve un error, el programa puede mostrar un mensaje en lugar de dejar la interfaz sin explicación. La API Fetch, por ejemplo, devuelve una promesa que permite procesar la respuesta cuando está disponible (MDN contributors, s. f.-b).

De manera resumida, el proceso sigue estos pasos:

- **Acción del usuario:** Se produce un evento dentro de la página.
- **Preparación de la petición:** JavaScript reúne la información y define la dirección y el método HTTP.
- **Envío al servidor:** El navegador realiza la solicitud de forma asíncrona.
- **Procesamiento:** El servidor recibe la petición, consulta o modifica los datos necesarios y prepara una respuesta.
- **Recepción de la respuesta:** El navegador entrega el resultado al código JavaScript.
- **Actualización de la página:** JavaScript interpreta la información y modifica solamente la sección necesaria.

Este modelo permite que la página responda a las acciones del usuario de una forma más fluida. La documentación de MDN explica que las solicitudes de red realizadas con

JavaScript permiten obtener recursos individuales y actualizar secciones de una página sin efectuar una navegación completa (MDN contributors, s. f.-a).

4. Relación entre AJAX, JavaScript, el navegador y el servidor.

AJAX funciona gracias a la colaboración de varios componentes que cumplen tareas diferentes. No existe una sola parte que realice todo el proceso; cada tecnología participa en una etapa de la comunicación.

- **JavaScript:** Es el lenguaje que controla la interacción. Detecta eventos, prepara las solicitudes, procesa las respuestas y modifica el contenido de la página. Puede utilizar XMLHttpRequest o la función fetch() para comunicarse con el servidor.
- **Navegador:** Es el entorno donde se ejecuta la página. Proporciona las API necesarias para realizar solicitudes HTTP, administra la comunicación de red y muestra el resultado. También aplica reglas de seguridad, como la política del mismo origen.
- **Servidor:** Recibe la petición, ejecuta la lógica de la aplicación, consulta o actualiza una base de datos cuando corresponde y devuelve una respuesta al navegador. Puede estar desarrollado con tecnologías como PHP, Python, Java, C# o Node.js.
- **HTML y DOM:** HTML define la estructura de la página y el DOM representa sus elementos como objetos que JavaScript puede consultar y modificar. Gracias a ello, el contenido recibido puede incorporarse en una sección específica.
- **HTTP y los datos:** HTTP establece cómo se envían las solicitudes y las respuestas. Los datos pueden transmitirse como JSON, XML, texto u otro formato que ambas partes hayan acordado.

Por ejemplo, cuando una persona solicita los datos de un usuario, JavaScript puede ejecutar fetch() desde el navegador. El servidor procesa la dirección solicitada y devuelve un documento JSON. Luego JavaScript lee los campos de ese documento y los coloca en los elementos HTML correspondientes. La API Fetch utiliza objetos Request y Response y permite configurar aspectos de la solicitud, como el método HTTP y los encabezados (MDN contributors, s. f.-b).

Es importante considerar que, si la página y el servicio se encuentran en orígenes diferentes, el navegador aplica las reglas de seguridad CORS. En esos casos, el servidor debe autorizar el origen que intenta acceder a la respuesta mediante los encabezados adecuados. CORS no es una falla de AJAX, sino un mecanismo de seguridad que controla el acceso a recursos entre distintos orígenes (MDN contributors, s. f.-d).

5. Ventajas y desventajas del uso de AJAX.

Ventajas:

- **Mayor rapidez percibida:** Al actualizar solo una parte de la página, el usuario no tiene que esperar a que se descargue nuevamente todo el documento.
- **Mejor experiencia de usuario:** La interfaz puede responder a las acciones sin interrumpir continuamente la navegación.
- **Menor transferencia de información en ciertas operaciones:** Se pueden solicitar únicamente los datos necesarios para la sección que se desea actualizar.
- **Interfaz dinámica:** Facilita búsquedas instantáneas, filtros, notificaciones, validaciones y paneles que cambian según los datos.
- **Comunicación flexible:** Puede utilizar diferentes métodos HTTP y formatos de respuesta, según las necesidades de la aplicación.
- **Compatibilidad con tecnologías modernas:** Los navegadores actuales ofrecen Fetch y XMLHttpRequest para realizar solicitudes desde JavaScript.

Desventajas:

- **Dependencia de JavaScript:** Si el código está desactivado o presenta errores, algunas funciones basadas en AJAX pueden dejar de trabajar.
- **Manejo adicional de errores:** El desarrollador debe contemplar problemas de conexión, respuestas inválidas, tiempos de espera y errores del servidor.
- **Complejidad del código:** Las solicitudes asíncronas requieren organizar correctamente las operaciones y controlar cuándo llegan las respuestas.
- **Restricciones de seguridad:** Las solicitudes entre diferentes orígenes están sujetas a la política del mismo origen y a la configuración CORS.
- **Accesibilidad y navegación:** Si la interfaz cambia sin actualizar correctamente el foco, el historial o los avisos para tecnologías de asistencia, puede resultar difícil de utilizar.
- **Rendimiento mal administrado:** Hacer demasiadas solicitudes o descargar datos innecesarios puede aumentar el consumo de recursos y afectar la experiencia.

En conclusión, AJAX ofrece beneficios importantes cuando se utiliza de forma adecuada, pero no garantiza por sí solo que una aplicación sea rápida, segura o accesible. Es necesario diseñar las solicitudes, validar la información en el servidor, controlar los errores y ofrecer señales claras sobre el estado de cada operación.

6. Usos actuales de AJAX.

AJAX se utiliza en diferentes funciones de las aplicaciones web modernas. Aunque el término se originó cuando era común intercambiar XML, hoy las solicitudes asíncronas suelen transportar JSON u otros formatos. La técnica continúa siendo útil porque permite separar la carga inicial de una página de las operaciones posteriores que necesitan información del servidor.

Formularios:

Cuando un usuario completa un formulario, JavaScript puede enviar sus datos al servidor sin abandonar la página. El servidor procesa la información y devuelve una confirmación o una lista de errores. De esta manera, el usuario puede corregir los campos señalados sin tener que comenzar nuevamente el proceso.

Búsquedas y filtros:

En una tienda virtual, por ejemplo, una persona puede escribir el nombre de un producto y recibir resultados relacionados mientras escribe. JavaScript envía el término de búsqueda, el servidor consulta los registros y devuelve una lista. La página sustituye los resultados anteriores por los nuevos sin recargar todo el sitio.

Validaciones:

AJAX permite consultar al servidor para comprobar información que requiere acceso a los datos de la aplicación. Un ejemplo sería verificar si un nombre de usuario ya está registrado. Esta comprobación complementa las validaciones realizadas en el navegador, pero no las reemplaza: el servidor debe validar nuevamente los datos antes de guardarlos.

Carga de datos:

Una página puede cargar comentarios, perfiles, productos, publicaciones o registros adicionales cuando el usuario los solicita. También puede incorporar nuevos elementos al desplazarse por una lista extensa. Así se evita descargar desde el principio información que quizá el usuario no llegue a consultar.

Aplicaciones web dinámicas:

Los paneles administrativos, sistemas de seguimiento, plataformas educativas y aplicaciones de comercio electrónico pueden utilizar solicitudes asíncronas para actualizar indicadores, notificaciones, estados de pedidos y otros datos. Estas operaciones permiten que la interfaz se mantenga actualizada mientras el usuario trabaja.

La documentación de MDN presenta las solicitudes de red como una práctica habitual para obtener recursos individuales y actualizar partes de sitios basados en datos. También señala que los datos solicitados pueden ser JSON, recursos multimedia, HTML o texto, según la finalidad de la aplicación (MDN contributors, s. f.-a).

7. Diferencia entre una página web tradicional y una página que utiliza AJAX.

En una página web tradicional, una acción que necesita información nueva suele provocar una solicitud de navegación. El servidor responde enviando un documento HTML completo y el navegador carga nuevamente la página. Este modelo continúa siendo válido y apropiado para numerosos sitios, pero puede implicar volver a descargar elementos que no cambiaron.

En una página que utiliza AJAX, JavaScript puede enviar una solicitud independiente para obtener o enviar únicamente la información necesaria. Cuando llega la respuesta, actualiza la parte correspondiente del documento y conserva el resto de la interfaz. La diferencia principal está en la forma de realizar las actualizaciones, no en que una página sea necesariamente más moderna o mejor que otra.

Aspecto	Página tradicional	Página con AJAX
Actualización	Generalmente solicita y carga un documento completo.	Puede actualizar solo la sección que necesita nuevos datos.
Comunicación	Suele realizarse mediante una navegación o envío tradicional del formulario.	JavaScript realiza solicitudes HTTP en segundo plano.
Continuidad	La navegación puede interrumpir momentáneamente la interacción.	El usuario puede continuar usando otras partes mientras espera.
Datos recibidos	Con frecuencia recibe una página HTML completa.	Puede recibir JSON, XML, texto o HTML parcial.
Uso común	Sitios informativos y páginas donde cada navegación carga un documento nuevo.	Búsquedas dinámicas, filtros, formularios y paneles actualizados.

La comparación muestra que AJAX puede reducir las recargas completas y hacer más fluida la interacción cuando solo se necesita cambiar una parte de la información. Sin embargo, ambos modelos pueden combinarse en un mismo proyecto. La elección depende del objetivo, la estructura del sitio, la accesibilidad y las necesidades de funcionamiento (MDN contributors, s. f.-a).

8. Ejemplo sencillo donde se explique el funcionamiento de AJAX.

Para comprender cómo trabaja AJAX, se presenta un ejemplo de una página que permite consultar los datos de un usuario al presionar un botón. La página enviará una solicitud GET a un servicio de prueba llamado JSONPlaceholder, que devuelve información ficticia en formato JSON. Cuando el navegador recibe la respuesta, JavaScript mostrará el nombre y el correo electrónico dentro de una sección de la misma página.

El ejemplo utiliza la función fetch(), que permite realizar solicitudes HTTP desde JavaScript. Esta función devuelve una promesa; por eso se utiliza async/await para esperar la respuesta y procesar los datos cuando estén disponibles. También se comprueba

response.ok para detectar respuestas HTTP que indiquen un error, ya que recibir una respuesta no significa necesariamente que la operación haya sido exitosa (MDN contributors, s. f.-b).

Código completo (archivo index.html):

```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Ejemplo de AJAX</title>
</head>
<body>
<h1>Consulta de usuario con AJAX</h1>
<button id="consultar">Consultar usuario</button>
<p id="estado">Presiona el boton para consultar los datos.</p>
<section id="resultado"></section>


<script>
const boton = document.getElementById("consultar");
const estado = document.getElementById("estado");
const resultado = document.getElementById("resultado");

boton.addEventListener("click", cargarUsuario);

async function cargarUsuario() {
estado.textContent = "Consultando al servidor...";
resultado.innerHTML = "";

try {
const respuesta = await fetch(
"https://jsonplaceholder.typicode.com/users/1"

);

if (!respuesta.ok) {
throw new Error("No se pudo obtener la informacion.");

}

const usuario = await respuesta.json();
resultado.innerHTML = `
<h2>${usuario.name}</h2>
<p>Correo: ${usuario.email}</p>
<p>Ciudad: ${usuario.address.city}</p>

;

estado.textContent = "Datos recibidos correctamente.";
} catch (error) {
estado.textContent = "Ocurrio un error al consultar los datos.";
console.error(error);

}

}
```

```
</script>
</body>
</html>
```

Explicación del funcionamiento:

- **Estructura HTML:** Se crea un botón para iniciar la consulta, un párrafo para mostrar el estado y una sección donde aparecerán los datos.
- **Selección de elementos:** JavaScript obtiene referencias al botón, al mensaje de estado y al espacio destinado para el resultado.
- **Evento click:** Cuando el usuario presiona el botón, se ejecuta la función cargarUsuario().
- **Solicitud fetch():** El navegador solicita el recurso /users/1 al servidor de prueba mediante el método GET.
- **Respuesta JSON:** Si la respuesta es correcta, response.json() convierte el contenido recibido en un objeto que JavaScript puede utilizar.
- **Actualización del DOM:** El programa coloca el nombre, el correo y la ciudad del usuario dentro de la sección resultado, sin recargar el documento completo.
- **Manejo de errores:** Si ocurre un problema de conexión o la respuesta no es satisfactoria, se muestra un mensaje de error.

Para probarlo, se debe guardar el código en un archivo llamado index.html y abrirlo mediante un servidor local de desarrollo, por ejemplo, la extensión Live Server de Visual Studio Code. Es necesario contar con conexión a Internet porque la información se solicita a un servicio externo. Al presionar el botón, primero se mostrará el mensaje de consulta y luego aparecerán los datos devueltos por el servicio. Si el servicio no está disponible o la conexión falla, se mostrará el mensaje de error.

Este ejemplo demuestra la idea central de AJAX: La página realiza una solicitud asíncrona, recibe datos del servidor y modifica una parte de su contenido. La página permanece abierta durante el proceso y no necesita volver a cargar todos sus elementos para presentar el resultado.

CONCLUSIÓN

AJAX es una técnica importante en el desarrollo web porque permite que una aplicación se comunique con un servidor sin tener que recargar por completo la página cada vez que necesita información. Su funcionamiento combina JavaScript, las herramientas de comunicación del navegador, las solicitudes HTTP y la lógica que se ejecuta en el servidor. De esta manera, los datos pueden obtenerse o enviarse en segundo plano y utilizarse para actualizar una sección específica de la interfaz.

A lo largo de la investigación se identificó que AJAX tiene como propósito mejorar la interacción y hacer más eficientes ciertas operaciones de una aplicación. Entre sus usos se encuentran el envío de formularios, las búsquedas dinámicas, las validaciones que requieren consultar el servidor, la carga progresiva de datos y la actualización de paneles. También se comprobó que las respuestas no tienen que estar en XML, ya que actualmente es frecuente utilizar JSON, texto o contenido HTML.

Asimismo, se analizaron sus ventajas y desventajas. AJAX puede reducir recargas innecesarias y ofrecer una experiencia más fluida, pero exige controlar los errores, administrar las solicitudes asíncronas y respetar las medidas de seguridad del navegador, entre ellas las reglas CORS cuando se accede a recursos de otro origen. Por esta razón, su implementación debe acompañarse de validaciones, mensajes claros y una organización adecuada del código.

El ejemplo práctico permitió observar cómo una página solicita datos de un usuario mediante fetch(), recibe una respuesta JSON y presenta el resultado sin reconstruir el documento completo. En conclusión, comprender AJAX es útil para quienes estudian programación porque ayuda a entender cómo se comunican las aplicaciones web actuales y cómo se construyen interfaces capaces de responder a las acciones de los usuarios de manera dinámica.

REFERENCIAS BIBLIOGRÁFICAS

MDN contributors. (s. f.-a). Making network requests with JavaScript. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Network_requests

MDN contributors. (s. f.-b). Fetch API. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

MDN contributors. (s. f.-c). XMLHttpRequest API. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest_API

MDN contributors. (s. f.-d). Cross-Origin Resource Sharing (CORS). MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS